FPGA Experiment Report

Experiment 5 — PS/2 Keyboard Interface

Task: FPGA 5 — Keyboard (PS/2 interface)
Student 1: Mahmood Stitia **ID:** 214032682
Student 2: Saleh Khalil **ID:** 213310485

Part 1 — Status update

The table below lists the submitted files and their status. The status is one of *Done* (module and simulation written and passing), *Partially done* (started but not fully working — explained in the notes), or *Not started*.

File / Step	Status	Notes / comments
`Ps2_Interface.v`	Done	PS/2 receiver; testbench passes
`Ps2_Interface_tb.v`	Done	Self-checking; all scenarios pass
`Ps2_Display.v`	Done	7-seg + LED strobe (CDC to 100 MHz)
`Ps2_Top.v`	Done	Top level, FPGA pin wiring
`task1.xdc`	Done	Pin + clock constraints

Issues

Any errors present at the time of submission are described here (per module / simulation, with what the error means).

- `Ps2_Interface.v`:
- `Ps2_Interface_tb.v`:
- `Ps2_Display.v`:
- `Ps2_Top.v`:
- `task1.xdc`:

Part 2 — Simulations and answers

This part contains the simulation of `Ps2_Interface` (annotated waveforms) followed by the answers to the lab's questions. *(Per the PDF, only `Ps2_Interface.v` is simulated.)*

Simulation — Ps2_Interface

[paste screenshot here]

Annotated waveform of `Ps2_Interface_tb`: PS2Clk, rstn, PS2Data, scancode[7:0], keyPressed. Draw arrows on: the reset test, the first make-code, the typematic repeats, the extended key, and the release / re-press.

What the waveform shows (annotate the real screenshot with arrows):

1. **Reset test** — with `rstn=0` the outputs are cleared even while `PS2Clk` is idle (the async-reset advantage).
2. **First make-code** `0x2E` — after the stop-bit edge `scancode` becomes `2E` and `keyPressed` pulses exactly once.
3. **Typematic repeats of** `0x2E` — the same code is sent twice more; `scancode` stays `2E` and `keyPressed` does *not* pulse again.
4. **Extended key** `0xE0,0x5A` — the `0xE0` frame leaves `scancode` unchanged and produces no pulse; the trailing `0x5A` sets `scancode=5A` with one pulse.
5. **Release** `0xF0,0x5A` — neither frame pulses; the displayed code is held.
6. **Re-press** `0x5A` **after release** — pulses again (a new press after a release is a new event).

[paste screenshot here]

Zoom-in showing that the data is sampled on the negative edge *of PS2Clk (arrow on a falling edge where scancode / keyPressed react).*

Answer — Asynchronous reset (advantages & disadvantages)

A flip-flop can be reset *synchronously* (the reset is sampled only on the active clock edge) or *asynchronously* (the reset appears in the sensitivity list and acts the instant it is asserted):

`always @(posedge clk)`	synchronous: reset acts only on a clock edge
`always @(posedge clk or negedge rstn)`	asynchronous: reset acts immediately

	Asynchronous reset	Synchronous reset
Advantage	Resets the logic *immediately*, even when *no clock edge is available*. Critical in this lab because `PS2Clk` is idle most of the time.	Easy static-timing analysis; the reset is "just another data signal", no special routing, no recovery / removal checks.
Disadvantage	Reset *removal* (de-assertion) is dangerous: releasing it too close to a clock edge violates recovery / removal time and can cause **metastability**; needs a reset synchroniser and dedicated async-reset routing.	Needs a toggling clock to take effect — useless while the clock is idle; the reset pulse must be at least one clock period wide to be captured.

Why it matters here. The keyboard clock `PS2Clk` toggles only during a transmission and is idle (high) otherwise. A synchronous reset on that domain would not act while the clock is idle, so `Ps2_Interface` uses an **active-low asynchronous reset** `rstn` (see Figure 1 of the assignment: the push-button `reset` is inverted to `~reset`/`rstn`). The de-assertion hazard is handled on the fast side by a multi-FF synchroniser (see `Ps2_Display`).

Answer — PS/2 interface (frame, non-toggling clock, key-press detection)

The PS/2 keyboard is a device-driven serial interface on two wires: `PS2Clk` (clock generated *by the keyboard*, ≈10–17 kHz) and `PS2Data`. One byte is sent as an **11-bit frame, LSB first**, and the host samples `PS2Data` **on the falling edge of** `PS2Clk`:

Bit 0	Bits 1–8	Bit 9	Bit 10
Start (0)	Data D0..D7 (LSB first)	Parity (odd)	Stop (1)

Key facts used by the design:

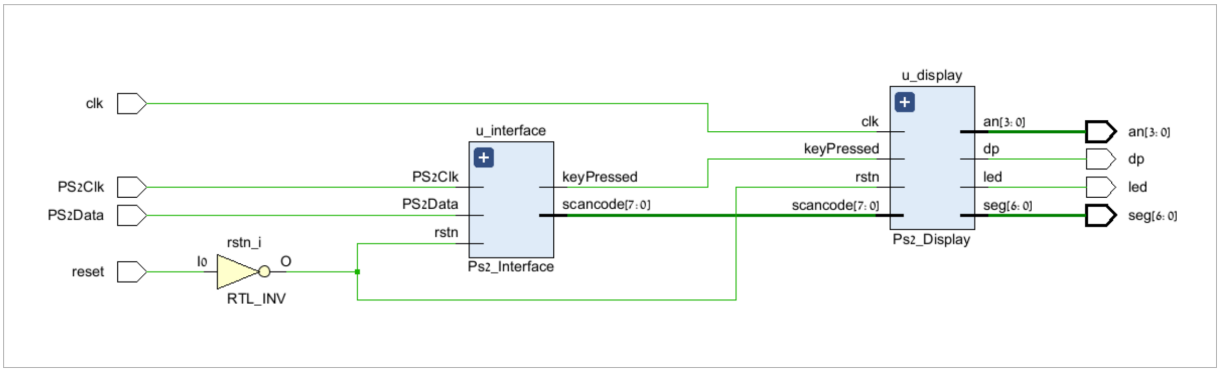
- **Make code** — sent on key press (e.g. keypad `5` → `0x2E`).
- **Typematic repeat** — while a key is held, the make code is re-sent repeatedly; `keyPressed` must pulse *only once* (on the first press).
- **Break code** — on release: `0xF0` followed by the make code.
- **Extended keys** — prefixed with `0xE0`; the `0xE0` must be ignored and only the trailing byte displayed.

Non-toggling clock ⇒ decode early. Because `PS2Clk` stops after the stop bit, the module must *not* wait for an edge after the frame. `Ps2_Interface` decodes the byte *on* the stop-bit edge (the last edge the keyboard actually produces), using a 22-bit shift register that holds the current frame and the previous one so two-frame sequences (`0xE0`/`0xF0`) can be recognised.

Two clock domains (CDC). `scancode`/`keyPressed` are produced in the slow `PS2Clk` domain and consumed in the fast 100 MHz `clk` domain of `Ps2_Display`. `keyPressed` is passed through a 3-FF synchroniser and edge-detected; `scancode` is captured on that already-settled pulse — a simple, safe data + valid CDC handshake. This is also why the logic analyser cannot debug both domains in a single ILA core.

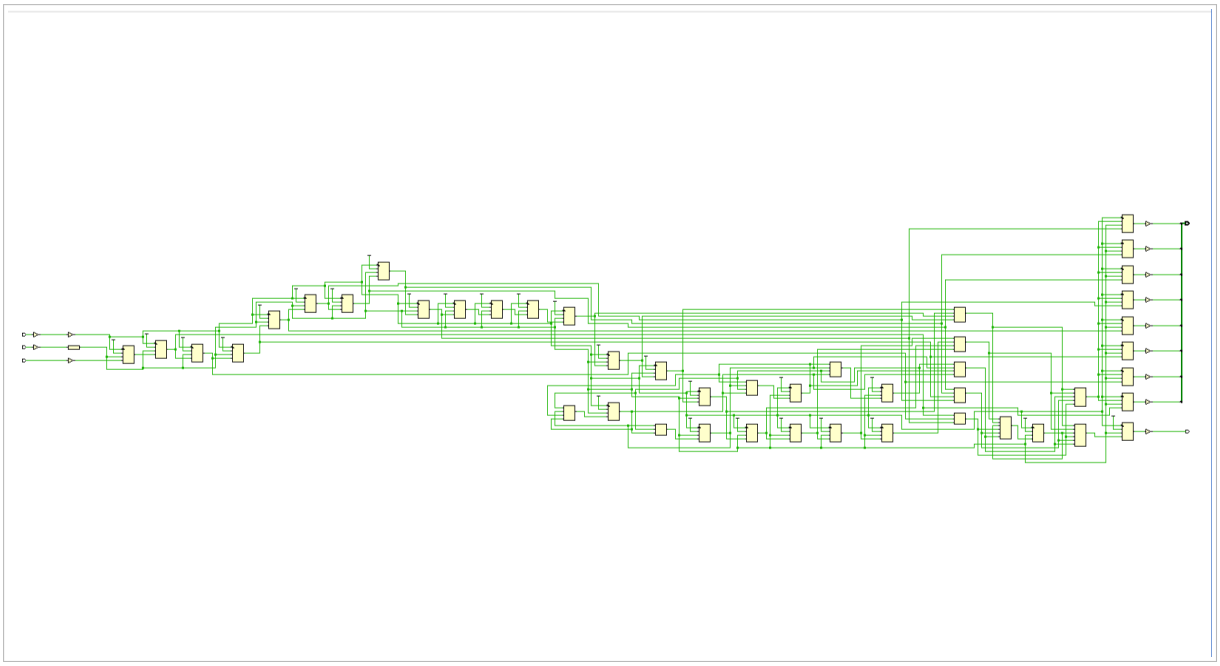
Elaborated Design

Top module



RTL-elaborated schematic of `Ps2_Top`: `u_interface` (`Ps2_Interface`, PS2Clk domain) feeds `keyPressed` and `scancode[7:0]` into `u_display` (`Ps2_Display`, 100 MHz domain); the `reset` push-button passes through an `RTL_INV` (`rstn_i`) into `rstn`. Inputs `clk`, `PS2Clk`, `PS2Data`, `reset` are on the left and outputs `an[3:0]`, `dp`, `led`, `seg[6:0]` on the right.

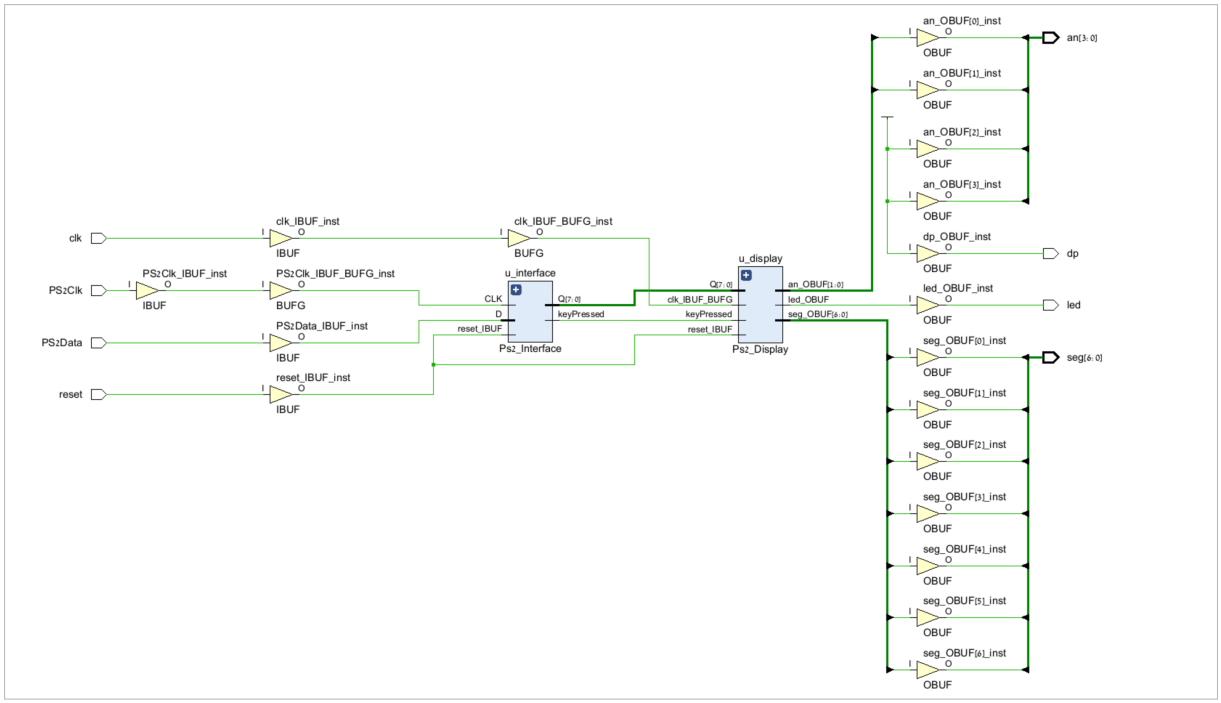
Ps2_Interface



RTL-elaborated schematic of `Ps2_Interface`: the 22-bit shift register and the `bitcount` counter feed the `held_code` / `key_down` registers and the make / break / extend decode logic, all clocked by `PS2Clk`; the outputs `scancode[7:0]` and `keyPressed` leave on the right.

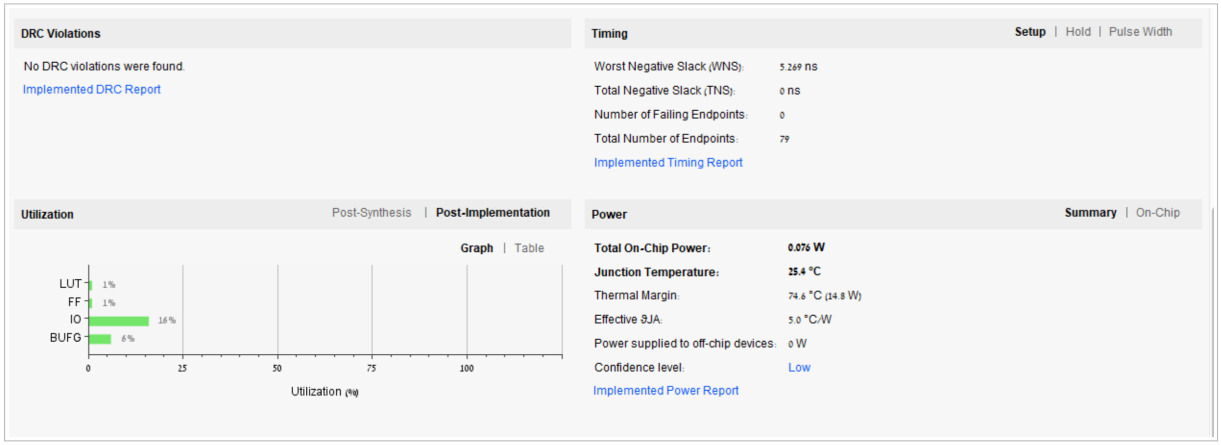
Implementation

Synthesis schematic



Post-synthesis schematic. Every external port now passes through an `IBUF/OBUF`, the two clocks (`clk`, `PS2Clk`) go through `BUFG` global buffers, and `u_interface` / `u_display` drive the `an`, `dp`, `led` and `seg` output buffers.

Project summary



Post-implementation project summary. No DRC violations; timing met (WNS $= 5.269$ ns, 0 failing endpoints); utilization is small (LUT & FF $\approx$1%, IO 16%, BUFG 6%); total on-chip power is 0.076 W with a junction temperature of 25.4°C.

Timing summary

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	5.269 ns	Worst Hold Slack (WHS):	0.330 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	79	Total Number of Endpoints:	79	Total Number of Endpoints:	57

All user specified timing constraints are met.

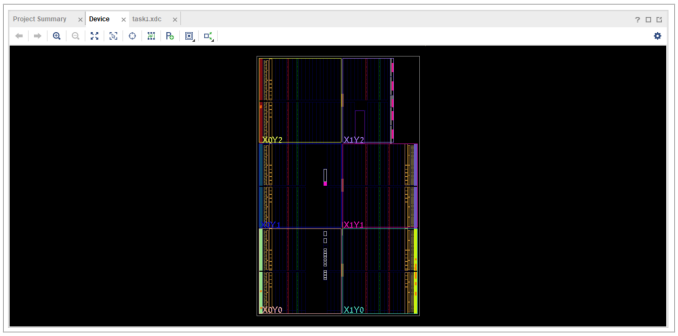
Design Timing Summary after implementation. The Setup (WNS $= 5.269$ ns), Hold (WHS $= 0.330$ ns) and Pulse-Width (WPWS $= 4.500$ ns) slacks are all positive with 0 failing endpoints — "All user specified timing constraints are met".

Critical Path

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock U
Path 1	5.269	3	25	u_display/led_timer_reg[16]/C	u_display/led_timer_reg[20]/CE	4.361	1.054	3.307	10.0	sys_clk_pin	sys_clk_pin		
Path 2	5.269	3	25	u_display/led_timer_reg[16]/C	u_display/led_timer_reg[21]/CE	4.361	1.054	3.307	10.0	sys_clk_pin	sys_clk_pin		
Path 3	5.269	3	25	u_display/led_timer_reg[16]/C	u_display/led_timer_reg[22]/CE	4.361	1.054	3.307	10.0	sys_clk_pin	sys_clk_pin		
Path 4	5.269	3	25	u_display/led_timer_reg[16]/C	u_display/led_timer_reg[23]/CE	4.361	1.054	3.307	10.0	sys_clk_pin	sys_clk_pin		
Path 5	5.534	3	25	u_display/led_timer_reg[16]/C	u_display/led_timer_reg[4]/CE	4.100	1.054	3.046	10.0	sys_clk_pin	sys_clk_pin		
Path 6	5.534	3	25	u_display/led_timer_reg[16]/C	u_display/led_timer_reg[5]/CE	4.100	1.054	3.046	10.0	sys_clk_pin	sys_clk_pin		
Path 7	5.534	3	25	u_display/led_timer_reg[16]/C	u_display/led_timer_reg[6]/CE	4.100	1.054	3.046	10.0	sys_clk_pin	sys_clk_pin		
Path 8	5.534	3	25	u_display/led_timer_reg[16]/C	u_display/led_timer_reg[7]/CE	4.100	1.054	3.046	10.0	sys_clk_pin	sys_clk_pin		
Path 9	5.576	3	25	u_display/led_timer_reg[16]/C	u_display/led_timer_reg[16]/CE	4.087	1.054	3.033	10.0	sys_clk_pin	sys_clk_pin		
Path 10	5.576	3	25	u_display/led_timer_reg[16]/C	u_display/led_timer_reg[17]/CE	4.087	1.054	3.033	10.0	sys_clk_pin	sys_clk_pin		

Intra-Clock Paths report. The worst paths (Path 1–4) run inside `u_display` from `led_timer_reg[16]/C` to `led_timer_reg[20..23]/CE`, with slack 5.269 ns, 3 logic levels and 4.361 ns total delay (1.054 ns logic $+ 3.307$ ns net) against the 10 ns requirement — comfortably met.

Device



Device (floorplan) view of the placed-and-routed design on the Artix-7 (`XC7A35T`). The few used slices are highlighted across the clock regions (`X0Y0`–`X1Y2`); the design occupies only a tiny fraction of the fabric.

Problems we faced

On hardware the design behaves as specified: pressing a numeric-keypad key shows its 8-bit make-code as two hex symbols on the two right-hand 7-segment digits (e.g. $5 \rightarrow 2\mathrm{E}$); each new press blinks the LED once with a short strobe (holding the key does *not* re-blink); the displayed code latches until the next press; and extended keys show only their last two hex symbols (the $0\mathrm{xE0}$ prefix is invisible).

(Describe the problems you actually hit in the lab and how you solved them. For example:)

- *(fill in)* where to put the `0xF0`/`0xE0` / two-key handling logic — in the interface vs. the display module — and the wrong display behaviour seen before moving it all into `Ps2_Interface`.
- *(fill in)* the keyboard clock being idle most of the time (decode before the stop bit, async reset).
- *(fill in)* CDC glitches on `keyPressed` and debugging the two clock domains in separate ILA cores.

[paste screenshot here]

Board photo: keypad plugged into the USB host port, a key pressed and its hex code shown on the two right digits with the LED lit.

Appendix — Verilog Source Code

The full source of the submitted files is listed below. Paste the code of each file inside its block.

Ps2_Interface.v

```
1 // --- paste Ps2_Interface.v here ---
```

Ps2_Interface_tb.v

```
// --- paste Ps2_Interface_tb.v here ---
```

Ps2_Display.v

```
// --- paste Ps2_Display.v here ---
```

Seg_7_Display.v

```
// --- paste Seg_7_Display.v here ---
```

Ps2_Top.v

```
// --- paste Ps2_Top.v here ---
```

task1.xdc

```
## --- paste task1.xdc here ---
```